

Rapport sur Jeu Pokémon

en Node.js

Réalisé par :
TAFTAF Wijdane

Encadré par : Pr. Amal Ourdo

Année Universitaire : 2025 / 2026

Table des matières

Introduction	2
0.1 Objectifs	2
0.2 Technologies et outils	2
Structure du Code	3
0.3 1. Définition des Pokémon	3
0.4 2. Fonctions utilitaires	3
0.4.1 Fonction d'attaque	3
0.5 3. Fonction principale	4
0.6 4. Test du jeu	4
0.7 Résumé des fonctions principales	5
Conclusion	8

Introduction

Ce TP consiste à développer un petit jeu de combat Pokémon exécuté dans le terminal à l'aide de Node.js. L'objectif est de mettre en pratique la programmation JavaScript côté serveur et d'utiliser des bibliothèques pour créer une interface interactive.

0.1 Objectifs

- Comprendre l'utilisation de Node.js pour exécuter des programmes JavaScript en dehors du navigateur.
- Manipuler des tableaux et objets en JavaScript.
- Développer des fonctions pour gérer la logique d'un combat Pokémon.
- Utiliser la bibliothèque **Inquirer** pour créer une interface interactive dans le terminal.
- Apprendre à gérer les entrées utilisateurs de manière asynchrone.

0.2 Technologies et outils

- **Node.js** : environnement d'exécution JavaScript côté serveur.
- **Inquirer** : bibliothèque pour créer des menus interactifs dans le terminal.

Structure du Code

0.3 1. Définition des Pokémon

On crée un tableau d'objets représentant les Pokémon, chacun ayant un nom, des points de vie (HP) et des attaques (moves) avec puissance, précision et points d'utilisation (PP).

```
import inquirer from "inquirer";
const pokemons = [{
  name: "Pikachu",
  hp: 250,
  moves: [
    { name: "Éclair", power: 40, accuracy: 90, pp: 5 },
    { name: "Queue de fer", power: 50, accuracy: 85, pp: 5 },
    { name: "Tonnerre", power: 90, accuracy: 70, pp: 3 },
    { name: "Vive-Attaque", power: 30, accuracy: 95, pp: 8 },
    { name: "Fatal-Foudre", power: 110, accuracy: 60, pp: 2 },
  ],
}, {
  name: "Dracaufeu",
  hp: 280,
  moves: [
    { name: "Lance-Flammes", power: 90, accuracy: 85, pp: 5 },
    { name: "Griffe", power: 40, accuracy: 95, pp: 10 },
    { name: "Danse Flammes", power: 70, accuracy: 80, pp: 4 },
    { name: "Déflagration", power: 110, accuracy: 60, pp: 2 },
    { name: "Coud'Œil", power: 75, accuracy: 85, pp: 5 },
  ],
}, {
  name: "Bulbizarre",
  hp: 260,
  moves: [
    { name: "Fouet Lianes", power: 45, accuracy: 90, pp: 6 },
    { name: "Tranch'Herbe", power: 55, accuracy: 95, pp: 6 },
    { name: "Lance-Soleil", power: 120, accuracy: 60, pp: 2 },
    { name: "Charge", power: 35, accuracy: 100, pp: 10 },
    { name: "Bombe Beurk", power: 90, accuracy: 70, pp: 4 },
  ],
},
];
```

0.4 2. Fonctions utilitaires

Barre de vie

La fonction `healthBar` affiche visuellement la vie d'un Pokémon avec des blocs verts et rouges.

```
function healthBar(currentHp, maxHp, barLength = 10) {
  const filled = Math.max(0, Math.min(barLength, Math.round((currentHp / maxHp) * barLength)));
  const empty = barLength - filled;
  return "■".repeat(filled) + "■".repeat(empty);
}
```

0.4.1 Fonction d'attaque

La fonction `attack` gère l'application d'une attaque d'un Pokémon à un autre.

```
function attack(attacker, defender, move) {
  if (move.pp <= 0) {
    console.log(` ${move.name} n'a plus de PP !`);
    return;
  }

  move.pp--;
  console.log(` ⚡ ${attacker.name} utilise ${move.name} (${move.pp} PP restants)`);

  const hitChance = Math.random() * 100;
  if (hitChance > move.accuracy) {
    console.log(` L'attaque échoue !`);
    return;
  }

  const damage = Math.floor(move.power + Math.random() * 10 - 5);
  defender.hp -= damage;
  if (defender.hp < 0) defender.hp = 0;

  console.log(` ${defender.name} perd ${damage} HP !`);
  console.log(
    ` ${defender.name} [${healthBar(defender.hp, 250)}] ${defender.hp} HP restants`
  );
}
```

0.5 3. Fonction principale

La fonction main orchestre le jeu :

- Choix du Pokémon du joueur via Inquierer.
- Choix aléatoire du Pokémon du bot.
- Boucle de combat jusqu'à ce que l'un des Pokémon soit KO.
- Alternance entre les tours du joueur et du bot.

```
async function main() {
  console.log("===  MINI COMBAT POKÉMON  ===\n");

  // Choix du Pokémon par le joueur
  const { playerPokemon } = await inquierer.prompt([
    {
      type: "list",
      name: "playerPokemon",
      message: "Choisis ton Pokémon :",
      choices: pokemons.map((p) => p.name),
    },
  ]);

  const player = JSON.parse(
    JSON.stringify(pokemons.find((p) => p.name === playerPokemon))
  );

  const bot = JSON.parse(
    JSON.stringify(
      pokemons[Math.floor(Math.random() * pokemons.length)]
    )
  );

  console.log(`\n Ton Pokémon : ${player.name}`);
  console.log(` Le bot choisit : ${bot.name}\n`);
}
```

0.6 4. Test du jeu

- Lancer le programme dans le terminal : `node index.js`. - Vérifier que :

```
// Boucle du combat
while (player.hp > 0 && bot.hp > 0) {
  console.log(`\n❤️ ${player.name}: [${healthBar(player.hp, 250)}] ${player.hp} HP`);
  console.log(`💀 ${bot.name}: [${healthBar(bot.hp, 250)}] ${bot.hp} HP\n`);

  const { moveName } = await inquirer.prompt([
    {
      type: "list",
      name: "moveName",
      message: "Choisis ton attaque :",
      choices: player.moves.map(
        (m) => `${m.name} (${m.pp} PP)`
      ),
    },
  ]);

  const move = player.moves.find((m) =>
    moveName.startsWith(m.name)
  );

  if (move.pp <= 0) {
    console.log(`❌ ${move.name} n'a plus de PP ! Choisis une autre attaque.`);
    continue;
  }

  attack(player, bot, move);
}
```

- Le joueur peut choisir un Pokémon.
- Le bot choisit un Pokémon aléatoire.
- Les attaques fonctionnent, avec gestion des PP et précision.
- La barre de vie se met à jour correctement.
- Le jeu se termine quand un Pokémon est KO.

0.7 Résumé des fonctions principales

- `healthBar(currentHp, maxHp)` : affiche une barre de vie visuelle.
- `attack(attacker, defender, move)` : applique les dégâts d'une attaque en tenant compte des PP et de la précision.
- `main()` : gère le déroulement du combat, l'interaction avec le joueur et le bot.

```
if (bot.hp <= 0) {
  console.log(`🏆 ${bot.name} est KO ! Tu gagnes le combat !`);
  break;
}

// Tour du bot
const botMoves = bot.moves.filter((m) => m.pp > 0);
const botMove = botMoves[Math.floor(Math.random() * botMoves.length)];
console.log(`\n ${bot.name} prépare son attaque...`);
await new Promise((r) => setTimeout(r, 1000));
attack(bot, player, botMove);

if (player.hp <= 0) {
  console.log(` ${player.name} est KO ! ${bot.name} gagne le combat...`);
  break;
}

console.log("\n-----\n");
}

console.log("\n===  Fin du combat  ===");
}

// Lancer le jeu
main();
```

```
PS D:\IID3\JS\pokemon-game> node index.js
>>
===  MINI COMBAT POKÉMON  ===

✓ Choisis ton Pokémon : Bulbizarre

Ton Pokémon : Bulbizarre
Le bot choisit : Dracaufeu

♥ Bulbizarre: [ ██████████ ] 260 HP
💀 Dracaufeu: [ ██████████ ] 280 HP

✓ Choisis ton attaque : Tranch'Herbe (6 PP)
⚡ Bulbizarre utilise Tranch'Herbe (5 PP restants)
Dracaufeu perd 57 HP !
Dracaufeu [ ██████████ ] 223 HP restants

Dracaufeu prépare son attaque...
⚡ Dracaufeu utilise Lance-Flammes (4 PP restants)
L'attaque échoue !

-----

♥ Bulbizarre: [ ██████████ ] 260 HP
💀 Dracaufeu: [ ██████████ ] 223 HP

? Choisis ton attaque :
> Fouet Lianes (6 PP)
  Tranch'Herbe (5 PP)
  Lance-Soleil (2 PP)
  Charge (10 PP)
  Bombe Beurk (4 PP)
```


Conclusion

Ce TP a permis de découvrir Node.js et d'apprendre à exécuter des programmes JavaScript directement dans le terminal. Il a également offert l'occasion de manipuler des objets et des tableaux afin de représenter les Pokémon et leurs attaques, tout en renforçant la compréhension des notions fondamentales de fonctions, boucles et conditions en JavaScript. L'utilisation de la librairie externe `Inquirer` a permis de rendre le programme interactif, en facilitant la gestion des choix du joueur dans le terminal. Enfin, ce TP a permis d'appliquer la logique d'un jeu de combat simple, avec la gestion des tours, des points de vie et des attaques, offrant une expérience pratique de développement d'un mini-jeu.